PostgreSQL CDC 수집과 변경 이벤트 검증

2회차 · 120분
원천 설정부터 Kafka 메시지, Bronze 행, Parquet 파일까지

강사 맥미니 시연 · 학생 SQL·이벤트 예측 · 개인 환경 복습

오늘의 120분

시간	진행	학생 활동
00:00-00:10	복습 진행 공유	해본 것과 막힌 점
00:10-00:30	지난 시간의 저장 결과	Bronze 3이력 → Parquet
00:30-00:50	PostgreSQL·Debezium 설정	발행 범위 · 이전 값 · 소비 위치
00:50-00:55	휴식	5분
00:55-01:15	스냅샷과 이벤트 파서	r · c/u/d · tombstone
01:15-01:35	새 환불 예측과 실행	Kafka → Bronze · 실제 파싱
01:35-01:50	수집 구간 진단	어디까지 도착했는가
01:50-02:00	정리와 복습	개인 실행 · 다음 회차

복습하면서 어디까지 해보셨나요?

해본 것

환경을 띄웠는지,
어떤 명령을 실행했는지

확인한 결과

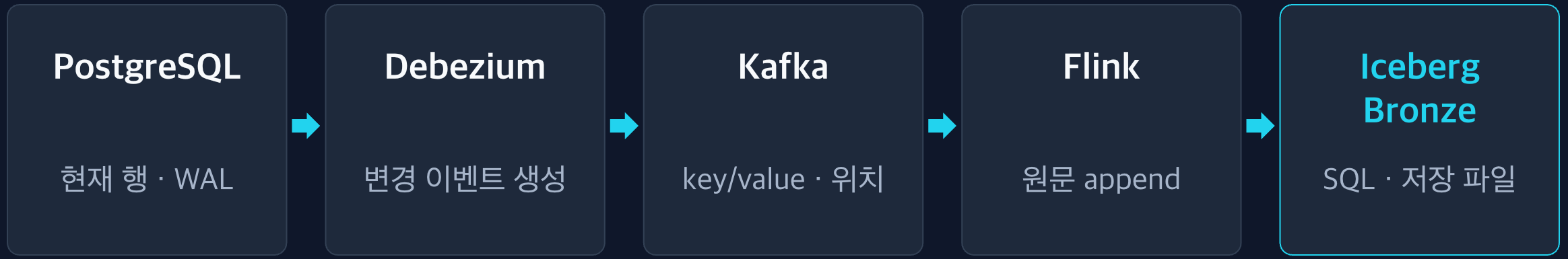
예상했던 값과
실제 결과가 같았는지

막힌 부분

오류 메시지와
다음에 시도할 것

SQL이나 실행 화면을 보면서 편하게 나눕니다

오늘 확인할 수집 경로



Spark SQL은 Bronze를 읽는 데 사용합니다
Silver·Gold 변환과 dbt 구현은 후속 회차입니다

삭제한 환불은 어디에 남을까요?

단계	원천 PostgreSQL	Kafka value / Bronze payload
INSERT	1행 · 20,000원	c · after.amount = 20000
UPDATE	1행 · 30,000원	u · before 20000 → after 30000
DELETE	0행	d · before 30000 → after null

같은 PK의 c/u/d는 서로 다른 변경 이력입니다

Bronze에서 같은 환불의 세 이력 조회

Spark SQL · lake.bronze.cdc_events

```
SELECT get_json_object(payload, '$.op') AS op,
       get_json_object(payload, '$.before.amount') AS before_amount,
       get_json_object(payload, '$.after.amount') AS after_amount,
       topic, partition_id, kafka_offset, _file
FROM lake.bronze.cdc_events
WHERE topic = 'cdc.public.refunds'
  AND (get_json_object(payload, '$.after.refund_id') = '<환불_ID>'
    OR get_json_object(payload, '$.before.refund_id') = '<환불_ID>')
ORDER BY partition_id, kafka_offset;
```

DELETE는 after가 null입니다. before의 PK도 조건에 포함합니다

조회 결과를 이렇게 대조합니다

확인할 것	판단할 근거
같은 환불인가	before / after의 refund_id
변경 내용이 맞는가	op와 20,000 → 30,000 → 삭제
어느 메시지인가	topic + partition_id + kafka_offset
어느 파일에 있는가	각 행의 _file 경로

오늘의 관찰은 이번 키의 결과입니다. 전체 데이터의 무누락 보장은 아닙니다

현재 테이블이 참조하는 파일 확인

Spark SQL · Iceberg files 메타데이터 테이블

```
SELECT file_path, file_format, record_count, file_size_in_bytes
FROM lake.bronze.cdc_events.files
WHERE content = 0
ORDER BY file_path;
```

_file 경로와 files.file_path를 연결해 실제 데이터 파일을 고릅니다

Parquet 파일의 내용까지 확인

Bronze SQL
대상 행 + _file

→

files 메타데이터
현재 파일 참조

→

MinIO 데이터 파일
실제 Parquet 읽기

→

내용 대조
payload · 위치

metadata.json이나 파일 이름을 보는 데서 끝내지 않습니다

파일 확인에서 구분할 것

변경 이벤트

c/u/d 각각이
Bronze의 한 이력

데이터 파일

여러 키·이벤트가
같이 들어갈 수 있음

Iceberg snapshot

테이블이 참조하는
파일 집합의 버전

DML 한 번 = 파일 한 개 = snapshot 한 개로 가정하지 않습니다

PostgreSQL에서 확인할 수집 조건

wal_level

논리 디코딩에 필요한
WAL 설정

publication

어떤 테이블의
변경을 발행할지

replica identity

UPDATE·DELETE에서
이전 행을 식별할 정보

replication slot

소비 진행 위치와
필요한 WAL 보존

이름이 비슷해도 역할은 다릅니다

publication과 수집 대상 테이블

PostgreSQL · app 데이터베이스

```
SHOW wal_level;

SELECT pubname, schemaname, tablename
FROM pg_publication_tables
WHERE pubname = 'cdc_lab'
ORDER BY schemaname, tablename;
```

현재 예제: public.orders · public.payments · public.refunds

이전 금액은 어떤 설정으로 관찰할까요?

PostgreSQL · 현재 설정 조회

```
SELECT c.relname, c.relreplident
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public'
  AND c.relname IN ('orders', 'payments', 'refunds');
```

이번 실습은 relreplident = f, REPLICA IDENTITY FULL입니다

slot의 두 위치와 WAL 보존

필드	읽을 의미
active	현재 slot을 스트리밍하는 소비자가 있는가
restart_lsn	slot이 아직 필요로 할 수 있는 가장 오래된 WAL 위치
confirmed_flush_lsn	논리 slot 소비자가 수신을 확인한 위치
wal_status	slot이 요구하는 WAL의 가용 상태

confirmed_flush_lsn은 Iceberg 적재 완료 위치가 아닙니다

Debezium 설정과 실제 수집 범위

실제 설정 중 학습에 필요한 항목만 표시

```
{
  "plugin.name": "pgoutput",
  "slot.name": "cdc_lab",
  "publication.name": "cdc_lab",
  "publication.autocreate.mode": "disabled",
  "table.include.list": "public.orders,public.payments,public.refunds",
  "topic.prefix": "cdc",
  "snapshot.mode": "initial",
  "tombstones.on.delete": "true"
}
```

publication · include list · 실제 이벤트를 함께 확인합니다

5분 쉬고 이어가겠습니다

돌아오면 스냅샷 r과 작은 파서로 이벤트를 해석합니다

초기 스냅샷 r과 이후 c/u/d

구분	무엇을 나타내는가	이번 시간의 확인
r	스냅샷에서 읽은 현재 행	보존된 실제 표본 또는 표시된 fixture
c	수집 시작 이후 생성 이벤트	새 환불 20,000원
u	기존 행의 변경 이벤트	같은 환불 30,000원으로 정정
d	물리 삭제 이벤트	before 값과 after=null

snapshot.mode=initial은 매 재시작마다 전체 스냅샷을 뜬다는 뜻이 아닙니다

작은 파서의 입력과 출력

입력

Kafka value 원문
+ key · topic · partition · offset

해석

JSON 구조와 op 확인
DELETE는 before에서 PK

출력

변경 요약 또는
tombstone · 오류 분류

원문 적재 파이프라인을 바꾸지 않는 독립적인 해석 도구입니다

변경 종류에 따라 행을 선택하기

Python · event_parser.py의 핵심 분기

```
# 전체 검증 코드는 event_parser.py에서 확인합니다.
if value is None:
    return {'kind': 'tombstone'}

event = json.loads(value)
op = event['op']
row = event.get('before') if op == 'd' else event.get('after')

# PK와 before/after 금액, 원천/전달 위치를 함께 해석합니다.
# 잘못된 JSON, 지원하지 않는 op, 필요한 행이 없는 경우는 구분합니다.
```

왜 DELETE에서 after를 선택하면 안 될까요?

파서 결과를 입력별로 확인

입력	확인할 것
스냅샷 r / 생성 c	after에서 PK와 현재 값
수정 u	before → after의 값 변화
삭제 d	before에서 PK, after=null 유지
wire value=null	tombstone으로 구분
잘못된 JSON / 구조	오류로 분류하고 정상 이벤트와 구분

이어서 새 환불을 실행하고 실제 메시지와 비교합니다

Kafka key · value · 수집 위치

key

{"refund_id": "...-r1"}
같은 원천 행을 식별

value

op · before · after · source
변경 내용과 원천 정보

수집 위치

topic · partition · offset
Kafka 레코드의 위치

Bronze는 value 원문과 위치를 저장합니다. key는 별도 컬럼이 없습니다

DELETE와 tombstone은 다릅니다

삭제 이벤트

value는 JSON 객체
op=d
before=삭제 전 행
after=null

tombstone

key는 그대로
wire value=null
이번 Bronze에서는 제외

Kafka 메시지 수와 Bronze 행 수를 그대로 같다고 비교하지 않습니다

새 환불을 실행하기 전에 예측

변경	원천 행 수	before.amount	after.amount
INSERT 20,000	?	?	?
UPDATE 30,000	?	?	?
DELETE	?	?	?

예측을 적고 실제 key · op · 값 · 위치와 비교합니다

원천 변경부터 Bronze 조회까지 대조

원천 변경
환불 c → u → d

➡

원천 SELECT
현재 행 확인

➡

Kafka 확인
key · value · 위치

➡

Bronze 확인
같은 위치와 내용

변경 직후 비어 있으면 도착·commit·조회 시점을 구분해 기다립니다

수집이 안 보일 때 확인할 구간

관찰	먼저 확인할 것
원천 행만 바뀜	commit · publication · include list · connector/task
Kafka에는 있고 Bronze에는 없음	topic 패턴 · Flink 처리 · checkpoint · sink commit
Bronze SQL과 파일이 다르게 보임	현재 파일 참조 · snapshot · 같은 키와 위치
변경이 중복처럼 보임	같은 PK의 다른 변경인지, 같은 전달 위치의 반복인지

“안 된다” 대신 마지막으로 확인한 지점을 말합니다

오늘 확인한 증거

수집 설정

publication · slot · FULL
Debezium include list

변경 이벤트

같은 환불의 c/u/d
key · 값 · 전달 위치

저장 결과

Bronze 세 이력
현재 Parquet 내용

설정이 맞는지와 결과가 맞는지를 함께 확인합니다

복습과 다음 회차

개인 환경 복습

공통 예제 실행
SQL·파서 결과·막힌 점 정리

제출과 공유

submission.md + SQL
진행한 과정을 동료와 공유

3회차

Flink → Bronze 적재 심화
개인 도메인·테이블 설계 확정

제출 방법·기한은 별도 안내를 따릅니다

공식 문서와 실습 자료

자료	확인할 내용
PostgreSQL 17	Logical decoding · publication · replication slots
Debezium 3.2	PostgreSQL connector · snapshot · change events
Apache Kafka 3.9	Partition ordering · log compaction
Apache Iceberg 1.10.0	Spark queries · metadata tables · Flink writes
Apache Spark 3.5.6	Parquet files
GitHub	kim-dabin/cdc-lakehouse-study · labs/session02

공식 링크는 발표자 노트와 labs/session02/references.md에 있습니다